Chapter 23

Machine Learning in Mass Spectrometry: High-Dimensional Regularization, Tree Ensembles, and Resampling Integrity

Lucas VHH TRAN[1]

Abstract

Multivariate predictive modeling in mass spectrometry aims to construct generalizable molecular signatures for patient stratification, disease diagnosis, and therapeutic response forecasting. Quantitative mass spectrometry operates in the high-dimensional regime where quantified features outnumber sample cohorts by one to three orders of magnitude ($p \gg n$; frequently $p \sim 10^3$--10^5 versus $n \sim 10^1$--10^2). In this geometry, ordinary least squares estimators are singular, and empirical classification accuracy can reach 100% on random noise through geometric over-fitting.

This chapter details the statistical foundations and computational workflows required to train and validate machine learning models in mass spectrometry. We examine the geometric properties of high-dimensional Euclidean space, the Curse of Dimensionality, and Cover's theorem on linear separability. We derive the analytical properties of penalized loss functions, focusing on the Elastic Net and its grouping bound for collinear peptide and metabolite clusters. We analyze non-linear ensemble architectures, including Random Forests and Gradient Boosted Decision Trees (XGBoost), compare Gini impurity against game-theoretic SHAP feature importance, and address Partial Least Squares Discriminant Analysis (PLS-DA). We establish resampling protocols to prevent data leakage, covering nested cross-validation, class-imbalance corrections, probability calibration (Brier score, Expected Calibration Error, and Platt scaling), and clinical decision curve analysis. The chapter provides an executable R laboratory using the `tidymodels` *framework, the Five Deadly Sins of omics machine learning, a diagnostic checklist, and worked exercises with solutions.*

Keywords: Machine Learning, Elastic Net, High-Dimensional Regularization, Random Forest, XGBoost, PLS-DA, SHAP Values, Data Leakage, Nested Cross-Validation, Probability Calibration, Decision Curve Analysis, Tidymodels.

[1]CK-CARE Davos, Christine Kühne -- Center for Allergy Research and Education, Davos, Switzerland. Email: `lucas.tran@ck-care.ch`

23.1 Tutor's Methodological Roadmap for Omics Machine Learning

Following the pedagogical framework of health analytics with R [4], translational mass spectrometry requires a multi-stage machine learning workflow before fitting predictive models. Table 23.1 summarizes the four checkpoints required to build verifiable predictive signatures.

Table 23.1: Tutor's Methodological Roadmap: Four Checkpoints for Machine Learning in Mass Spectrometry

Stage	Physical Mechanism	Key Analytical Objective	Primary Failure Mode & Remedy
1. Data Topology	High-dimensional space ($p \gg n$) causes distance concentration and empty space [10, 14].	Audit missingness, collinear blocks, class imbalance, and batch structures; stratify splits.	Pre-screening features or pooling batches across cohorts (data leakage). Remedy: Perform all filtering, imputation, and batch fitting inside CV folds [1, 7].
2. Model Architecture	Linear sparsity (Elastic Net [28]), latent projection (PLS-DA, OPLS-DA [2, 24]), vs. non-linear interactions (Random Forest, XGBoost [5, 9]).	Select inductive bias matching biological hypothesis; preserve collinear groups.	LASSO randomly dropping correlated peptides. Remedy: Elastic Net ($\alpha \in [0.2, 0.8]$) [28].
3. Resampling Integrity	Hyperparameter tuning on test splits produces optimistic performance bias [16].	Implement nested cross-validation (inner tuning loop, outer evaluation loop) or Leave-One-Batch-Out CV (LOBOCV).	Over-optimistic AUC reports. Remedy: Nested 5×5 cross-validation or LOBOCV with untouched holdout test partition [17].
4. Clinical Utility	Raw classifier scores are uncalibrated risk probabilities [6, 20].	Evaluate Brier score, Expected Calibration Error (ECE), and Decision Curve Analysis.	High AUC with distorted probabilities. Remedy: Platt scaling or isotonic calibration [26].

Methodological Note

Tutor's Methodological Warning: The Data Leakage Catastrophe

Data leakage occurs when information from outside the training dataset influences model training [1, 7]. If you normalize, impute missing values, or screen features via univariate two-sample tests (even with Benjamini-Hochberg false discovery rate control [3]) across all N patients before cross-validation splitting, information from the validation test fold leaks into the training pipeline. The resulting 98% accuracy is an artifact of leakage that drops toward chance on independent clinical cohorts. All preprocessing steps must remain encapsulated strictly within the resampling training fold [17].

23.2 The High-Dimensional Pathology ($p \gg n$) in Mass Spectrometry

Clinical proteomics cohorts quantify $p = 4{,}000$ to $10{,}000$ proteins, and untargeted metabolomics profiles thousands of spectral features across $n = 50$ to 200 patients. This condition violates the asymptotic assumptions of classical multivariate statistics.

23.2.1 The Curse of Dimensionality and Geometric Sparsity

As dimension p increases, Euclidean geometry exhibits two non-intuitive behaviors:

1. **Hypersphere Volume Vanishing:** The volume of a p-dimensional hypersphere of radius r inscribed within a hypercube of side length $2r$ is $V_p(r) = \frac{\pi^{p/2}}{\Gamma(p/2+1)} r^p$. The ratio of the sphere volume to the enclosing hypercube $(2r)^p$ converges asymptotically to zero: $\lim_{p\to\infty} \frac{V_p(r)}{(2r)^p} = \lim_{p\to\infty} \frac{\pi^{p/2}}{2^p \Gamma\left(\frac{p}{2}+1\right)} = 0$ [14]. Almost all volume concentrates in the corners of the hypercube.
2. **Distance Concentration:** For n points drawn independently in $\mathbb{R}^p$, pairwise distances concentrate around the mean: $\lim_{p\to\infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} = 0$. Sample pairs become equidistant, causing unpenalized nearest-neighbor algorithms to degenerate.

23.2.2 Cover's Theorem on Separability

Consider n random data points in general position in $\mathbb{R}^p$, each assigned a binary label $y_i \in \{-1, +1\}$ uniformly at random. By Cover's theorem [10], the probability $P(n, p)$ that these points are linearly separable by a hyperplane is:

$$P(n,p) = \left(\frac{1}{2}\right)^{n-1} \sum_{k=0}^{p} \binom{n-1}{k} \tag{23.1}$$

Whenever $p \geq n-1$, the summation encompasses all terms $0 \leq k \leq n-1$, giving $\sum_{k=0}^{n-1} \binom{n-1}{k} = 2^{n-1} \implies P(n,p) = 1.0$.

Biological Discovery & Validation

The Inevitability of 100% Training Accuracy:
When features outnumber samples ($p > n$), an unregularized linear classifier achieves 100% training accuracy on Gaussian noise with randomly permuted labels. High training accuracy in $p \gg n$ data is not evidence of biomarker discovery; it is a mathematical property of the linear system's null space [10, 25].

23.2.3 Batch Effects, Confounder Leakage, and Group-Aware Resampling

Instrument drift, column aging, ambient temperature variation, and reagent lots generate technical batch effects in mass spectrometry assays [7]. When batch normalization algorithms (such as ComBat or empirical Bayes scaling) are fitted across the pooled cohort before splitting, estimated location and scale parameters borrow variance across partition boundaries. This leaks test partition outcome information into the training data, inflating validation accuracy on pure noise. When biological classes correlate with acquisition batches or run order, classifiers memorize batch-specific background ions rather than disease markers. Preprocessing pipelines must estimate batch adjustment parameters strictly on the training partition and apply frozen transformations to validation folds. In multi-site or multi-day studies, models must be evaluated using Leave-One-Batch-Out Cross-Validation (LOBOCV) or stratified Group K-Fold cross-validation to assess true cross-batch generalization.

23.3 Theoretical Foundations of Mass Spectrometry Machine Learning

To prevent over-fitting in the $p \gg n$ regime, we introduce inductive biases through regularized loss functions, latent subspace projections, and non-linear tree ensembles.

23.3.1 Penalized Estimation: Ridge, LASSO, and the Elastic Net

For binary classification with labels $y_i \in \{0,1\}$ and standardized feature vector $\mathbf{x}_i \in \mathbb{R}^p$, the penalized logistic regression objective minimizes the empirical negative log-likelihood plus a penalty functional $\mathscr{P}_\alpha(\beta)$ [15, 23]:

$$\min_{\beta_0,\beta} \left\{ -\frac{1}{n}\sum_{i=1}^{n} [y_i \ln \pi(\mathbf{x}_i) + (1-y_i)\ln(1-\pi(\mathbf{x}_i))] + \lambda\, \mathscr{P}_\alpha(\beta) \right\} \tag{23.2}$$

where $\pi(\mathbf{x}_i) = \frac{1}{1+\exp(-(\beta_0+\mathbf{x}_i^T\beta))}$.

Zou and Hastie [28] formulated the Elastic Net penalty as a convex combination of ℓ_1 (LASSO [23]) and ℓ_2 (Ridge [15]) penalties:

$$\mathscr{P}_\alpha(\beta) = \alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2 = \sum_{j=1}^{p}\left(\alpha|\beta_j| + \frac{1-\alpha}{2}\beta_j^2\right) \tag{23.3}$$

where $\lambda > 0$ dictates shrinkage intensity, and $\alpha \in [0,1]$ governs penalty geometry ($\alpha = 0$ is Ridge; $\alpha = 1$ is LASSO; $0 < \alpha < 1$ is Elastic Net).

Because the logistic negative log-likelihood is non-quadratic, Friedman, Hastie, and Tibshirani [13] formulate coordinate descent within Penalized Iteratively Reweighted Least Squares (PIRLS). At each outer iteration, a two-term Taylor expansion around current parameter estimates $(\tilde{\beta}_0, \tilde{\beta})$ forms a weighted quadratic surrogate with working response z_i and observation weights w_i:

$$z_i = \tilde{\eta}_i + \frac{y_i - \pi(\mathbf{x}_i)}{\pi(\mathbf{x}_i)(1-\pi(\mathbf{x}_i))}, \quad w_i = \pi(\mathbf{x}_i)(1-\pi(\mathbf{x}_i)) \tag{23.4}$$

where $\tilde{\eta}_i = \tilde{\beta}_0 + \mathbf{x}_i^T\tilde{\beta}$ is the current linear predictor. Cyclical coordinate descent then updates each coordinate $j \in \{1,\ldots,p\}$ via the weighted soft-thresholding operator:

$$\hat{\beta}_j = \frac{\mathscr{S}\left(\frac{1}{n}\sum_{i=1}^{n} w_i x_{ij}(z_i - \tilde{z}_i^{(-j)}),\ \lambda\alpha\right)}{\frac{1}{n}\sum_{i=1}^{n} w_i x_{ij}^2 + \lambda(1-\alpha)} \tag{23.5}$$

where $\tilde{z}_i^{(-j)} = \tilde{\beta}_0 + \sum_{k\neq j} x_{ik}\tilde{\beta}_k$, and $\mathscr{S}(u,\gamma) = \operatorname{sign}(u)\max(0,|u|-\gamma)$. When the partial correlation $|u| \leq \lambda\alpha$, coefficient $\hat{\beta}_j$ is set identically to zero.

Mathematical Deep Dive

Mathematical Proof of the Elastic Net Grouping Bound:
Let $\hat{\beta}$ be the Elastic Net minimizer for normalized predictors $\mathbf{x}_j$ and $\mathbf{x}_k$ with sample correlation $\rho = \frac{1}{n}\mathbf{x}_j^T\mathbf{x}_k$. Assume $\operatorname{sign}(\hat{\beta}_j) = \operatorname{sign}(\hat{\beta}_k) > 0$. From the sub-gradient optimality conditions of Eq. (23.2):

$$-\frac{1}{n}\mathbf{x}_j^T(\mathbf{y} - \mathbf{X}\hat{\beta}) + \lambda\alpha + \lambda(1-\alpha)\hat{\beta}_j = 0 \tag{23.6}$$

$$-\frac{1}{n}\mathbf{x}_k^T(\mathbf{y} - \mathbf{X}\hat{\beta}) + \lambda\alpha + \lambda(1-\alpha)\hat{\beta}_k = 0 \tag{23.7}$$

Subtracting the second equality from the first:

$$\lambda(1-\alpha)(\hat{\beta}_j - \hat{\beta}_k) = \frac{1}{n}(\mathbf{x}_j - \mathbf{x}_k)^T(\mathbf{y} - \mathbf{X}\hat{\beta}) \tag{23.8}$$

Applying Cauchy-Schwarz with standardized vectors ($\|\mathbf{x}_j\|_2^2 = \|\mathbf{x}_k\|_2^2 = n$ and $\|\mathbf{x}_j - \mathbf{x}_k\|_2 = \sqrt{2n(1-\rho)}$):

$$|\hat{\beta}_j - \hat{\beta}_k| \leq \frac{\sqrt{2(1-\rho)}}{\lambda(1-\alpha)} \cdot \text{RMSE} \tag{23.9}$$

As correlation $\rho \to 1$, the upper bound converges to zero. Correlated biomarkers receive near-identical coefficients, preventing arbitrary feature drops [28].

23.3.2 Partial Least Squares Discriminant Analysis (PLS-DA, OPLS-DA) and the Overfitting Trap

Partial Least Squares Discriminant Analysis projects high-dimensional predictors $\mathbf{X} \in \mathbb{R}^{n \times p}$ onto a low-dimensional latent subspace that maximizes covariance with class labels $\mathbf{Y}$ [2]:

$$\max_{\mathbf{w},\mathbf{q}} \operatorname{cov}(\mathbf{X}\mathbf{w}, \mathbf{Y}\mathbf{q}) \quad \text{subject to } \|\mathbf{w}\|_2 = 1,\ \|\mathbf{q}\|_2 = 1 \tag{23.10}$$

Trygg and Wold [24] developed Orthogonal PLS-DA (OPLS-DA) to separate predictive from orthogonal variation:

$$\mathbf{X} = \mathbf{T}_p \mathbf{P}_p^T + \mathbf{T}_o \mathbf{P}_o^T + \mathbf{E} \tag{23.11}$$

where $\mathbf{T}_p \mathbf{P}_p^T$ isolates variation correlated with $\mathbf{Y}$, and $\mathbf{T}_o \mathbf{P}_o^T$ captures variation orthogonal to $\mathbf{Y}$, such as analytical drift or patient demographic covariates. In binary problems, OPLS-DA condenses the predictive variation onto a single score vector $\mathbf{t}_p$, enabling biomarker visualization through S-plots that map feature covariance against correlation.

Neither PLS-DA nor OPLS-DA prevents geometric over-fitting in $p \gg n$ settings [7, 27]. Because $\mathbf{Y}$ guides the orthogonal projection, OPLS-DA strips non-predictive variance out of the primary component, separating pure Gaussian noise into distinct visual clusters. The goodness-of-fit metric R_Y^2 is inflated by noise. Models must be validated using cross-validated predictive ability (Q_Y^2) and non-parametric response permutation testing ($N \geq 1{,}000$ label permutations) [27].

23.3.3 Tree Ensembles: Random Forest and Gradient Tree Boosting

When biological relationships involve non-linear thresholds or epistatic interactions, decision tree ensembles provide effective predictive performance:

- **Random Forest:** Breiman [5] averages B de-correlated decision trees constructed via bootstrap aggregation (bagging) and random feature sub-sampling ($\sqrt{p}$ candidate features per split). For individual tree variance σ^2 and pairwise tree correlation ρ, ensemble variance is:

$$\operatorname{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{23.12}$$

  As $B \to \infty$, variance converges to $\rho\sigma^2$. Random feature selection reduces ρ, lowering overall prediction variance without increasing bias.

- **Gradient Tree Boosting (XGBoost):** Chen and Guestrin [9] extended Friedman's gradient boosting framework [12] by optimizing a regularized objective via second-order Taylor expansion. At boosting round t, the objective for tree f_t with T_{leaves} terminal regions and leaf weights $\mathbf{w} \in \mathbb{R}^{T_{\text{leaves}}}$ is:

$$\tilde{\mathscr{L}}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma T_{\text{leaves}} + \frac{\lambda}{2} \sum_{j=1}^{T_{\text{leaves}}} w_j^2 \tag{23.13}$$

  where $f_t(\mathbf{x}_i) = w_{q(\mathbf{x}_i)}$ with leaf assignment function $q : \mathbb{R}^p \to \{1, \ldots, T_{\text{leaves}}\}$, $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$, and $h_i = \partial^2_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$. Regularization parameter γ penalizes leaf proliferation, and λ shrinks leaf weights.

In untargeted metabolomics feature tables ($p = 20{,}000$ to $100{,}000$), evaluating $m_{\text{try}} \approx \sqrt{p}$ candidate features per split encounters a split candidate dilution problem. When true biomarkers are sparse, candidate subsets rarely contain informative features, forcing individual decision trees to split on noise. This inflates

individual tree error variance σ^2 and degrades ensemble accuracy. Exact TreeSHAP computes Shapley values in polynomial time $O(T_{\text{trees}} L_{\text{leaves}} D^2)$, where D is tree depth and L_{leaves} is maximum leaf count [19]. In contrast, cyclical coordinate descent for Elastic Net requires $O(np)$ operations per full coordinate sweep across all p features, making penalized linear models efficient for high-dimensional screening. When non-linear ensembles are required on massive datasets, unsupervised variance filtering or principal component extraction must be executed strictly inside the resampling recipe before tree construction.

23.3.4 Interpretability: Gini Impurity Selection Bias vs Game-Theoretic SHAP

In clinical biomarker translation, identifying feature contributions is essential:

- **Gini Impurity Bias:** Classical Random Forest variable importance measures the total decrease in node impurity (Gini index). Strobl et al. [22] proved that Gini importance is biased toward continuous features with high variance and multiple split points, over-ranking noise variables.
- **Game-Theoretic SHAP:** Lundberg and Lee [18] applied cooperative game theory to model interpretation. The Shapley value ϕ_j assigns a unique additive attribution to feature j across all possible feature subsets $S \subseteq F \setminus \{j\}$:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[f(S \cup \{j\}) - f(S) \right] \tag{23.14}$$

  TreeSHAP computes exact Shapley attributions in polynomial time $O(TLD^2)$, satisfying efficiency, symmetry, dummy, and additivity axioms [19].

23.3.5 Probability Calibration and Clinical Decision Curve Analysis

Clinical decisions depend on calibrated disease probabilities $\hat{p}_i \in [0,1]$ rather than hard binary labels:

- **Brier Score and Calibration:** The Brier score measures mean squared probability error:

$$\text{BS} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{p}_i)^2 \tag{23.15}$$

  Under Murphy's decomposition, $\text{BS} = \text{Reliability} - \text{Resolution} + \text{Uncertainty}$. Platt scaling fits a sigmoid transformation to map uncalibrated scores to empirical probabilities [20]:

$$\hat{p}_i^{\text{cal}} = \frac{1}{1 + \exp(A \hat{f}_i + B)} \tag{23.16}$$

- **Decision Curve Analysis (DCA):** Vickers and Elkin [26] introduced net benefit analysis:

$$\text{Net Benefit}(p_t) = \frac{\text{True Positives}}{N} - \frac{\text{False Positives}}{N} \left(\frac{p_t}{1 - p_t} \right) \tag{23.17}$$

  where p_t is the clinical decision threshold probability. DCA directly benchmarks the clinical net utility of the mass spectrometry signature against treat-all and treat-none strategies across clinical thresholds.

23.4 Hands-On R Laboratory: Tidymodels Workflow in Proteomics

We now implement an end-to-end, multi-model machine learning pipeline using the modern `tidymodels` framework [17]. All code uses base R pipe syntax (`|>`) and is fully reproducible.

R Lab Listing 1: Simulating High-Dimensional Proteomics Cohort ($p \gg n$) with MAR and Batch Effect [R 4.6 / Bioconductor]

```r
suppressPackageStartupMessages({
  library(tidymodels)
  library(glmnet)
  library(ranger)
  library(xgboost)
  library(themis)
  library(probably)
  library(dcurves)
  library(dplyr)
  library(ggplot2)
})

set.seed(2026)
n_samples  <- 120
n_proteins <- 4000L

status <- factor(c(rep("Control", 80), rep("Tumor", 40)), levels = c("Control", "Tumor"))

# Observed continuous covariate: Age drives MAR missingness
age_years <- c(rnorm(80, mean = 55, sd = 12), rnorm(40, mean = 58, sd = 12))
age_years <- pmax(18, pmin(90, age_years))

# Confounded acquisition batch assignment
assign_batch <- function(status_vec, seed = 2026) {
  set.seed(seed)
  batches <- character(length(status_vec))
  for (i in seq_along(status_vec)) {
    probs <- if (status_vec[i] == "Control") c(0.40, 0.25, 0.18, 0.10, 0.07) else c(0.05, 0.10, 0.18, 0.27, 0.40)
    batches[i] <- sample(LETTERS[1:5], size = 1, prob = probs)
  }
  factor(batches, levels = LETTERS[1:5])
}
batch_vec <- assign_batch(as.character(status))

X <- matrix(rnorm(n_samples * n_proteins, mean = 20, sd = 1.0),
            nrow = n_samples, ncol = n_proteins)
colnames(X) <- paste0("PROT_", sprintf("%04d", 1:n_proteins))

# Module 1: Collinear co-regulated signature (Proteins 1-8, r ~ 0.70)
latent_driver <- c(rep(0, 80), rep(0.65, 40))
for (j in 1:8) {
  X[, j] <- latent_driver + rnorm(n_samples, mean = 20, sd = 0.8)
}

# Module 2: Epistatic interaction (Proteins 9 & 10, zero main effect)
tumor_idx <- 81:120
X[, 9]  <- X[, 9]  - 20
X[, 10] <- X[, 10] - 20
interaction_driver <- rnorm(40, mean = 1.5, sd = 0.3)
X[tumor_idx, 9]  <- X[tumor_idx, 9]  + 0.6 * interaction_driver
X[tumor_idx, 10] <- X[tumor_idx, 10] + 0.6 * interaction_driver

# Inject technical batch effects on non-marker background proteins (PROT_0021 to PROT_0220)
batch_shifts <- c(A = -0.75, B = -0.35, C = 0.00, D = 0.35, E = 0.75)
for (i in 1:n_samples) {
```

```
  b <- as.character(batch_vec[i])
  X[i, 21:220] <- X[i, 21:220] + batch_shifts[b]
}

# MAR missingness driven by Age
miss_prob <- plogis(-3.2 + 0.05 * (age_years - 55))
miss_mask <- matrix(runif(n_samples * n_proteins) < rep(miss_prob, times = n_proteins),
                    nrow = n_samples, ncol = n_proteins)

# MNAR component on low-abundance proteins near detection limit
low_abund <- which(colMeans(X, na.rm = TRUE) < 19.5)
for (j in low_abund) {
  thresh_j <- quantile(X[, j], 0.05, na.rm = TRUE)
  miss_mask[, j] <- miss_mask[, j] | ((X[, j] < thresh_j) & (runif(n_samples) < 0.30))
}
X[miss_mask] <- NA

df_cohort <- as.data.frame(X)
df_cohort$Status <- status
df_cohort$Age    <- age_years
df_cohort$Batch  <- batch_vec

# Stratified 75/25 Train-Test Split
set.seed(42)
split_obj  <- initial_split(df_cohort, prop = 0.75, strata = Status)
train_data <- training(split_obj)
test_data  <- testing(split_obj)

cat(sprintf("Simulated cohort: %d samples x %d features (p/n ratio ~ %.0f).\n",
            nrow(df_cohort), ncol(df_cohort) - 3, (ncol(df_cohort) - 3) / nrow(df_cohort)))
cat(sprintf("Training set: %d patients (%d Control, %d Tumor)\n",
            nrow(train_data), sum(train_data$Status == "Control"), sum(train_data$Status == "Tumor")))
cat(sprintf("Independent test holdout: %d patients (%d Control, %d Tumor)\n",
            nrow(test_data), sum(test_data$Status == "Control"), sum(test_data$Status == "Tumor")))

# MAR audit across age quartiles
train_data$.age_quartile <- cut(train_data$Age,
  breaks = quantile(train_data$Age, c(0, 0.25, 0.5, 0.75, 1)),
  include.lowest = TRUE, labels = c("Q1(young)", "Q2", "Q3", "Q4(old)"))
protein_cols <- grep("^PROT_", names(train_data), value = TRUE)
miss_by_age <- train_data |> group_by(.age_quartile) |>
  summarise(missingness_rate = round(mean(is.na(as.matrix(across(all_of(protein_cols))))), 4),
            n = n(), .groups = "drop")
print(miss_by_age)
train_data$.age_quartile <- NULL
```

```
Simulated cohort: 120 samples x 4000 features (p/n ratio   33).
Training set: 90 patients (60 Control, 30 Tumor)
Independent test holdout: 30 patients (20 Control, 10 Tumor)
  .age_quartile missingness_rate n
1 Q1(young) 0.0188 23
2 Q2 0.0313 22
3 Q3 0.0470 22
4 Q4(old) 0.0890 23
```

R Lab Listing 2: Leakage-Safe Preprocessing Recipe with Role Assignment [R 4.6 / Bioconductor]

```
# Define preprocessing recipe strictly on training split
rec <- recipe(Status ~ ., data = train_data) |>
  update_role(Batch, new_role = "batch_id") |>
  update_role(Age, new_role = "imputation donor") |>
  step_impute_median(all_numeric_predictors()) |>
  step_zv(all_numeric_predictors()) |>
  step_normalize(all_numeric_predictors()) |>
  step_downsample(Status, under_ratio = 1.2)

prep_rec <- prep(rec, training = train_data)
baked_train <- bake(prep_rec, new_data = NULL)

cat(sprintf("Recipe successfully baked on training fold:\n"))
cat(sprintf("  Predictors processed: %d (Zero-variance filtered: %d)\n",
            ncol(baked_train) - 3, ncol(train_data) - 3 - (ncol(baked_train) - 3)))
cat(sprintf("  Post-downsample class balance: %d Control, %d Tumor\n",
            sum(baked_train$Status == "Control"), sum(baked_train$Status == "Tumor")))
cat(sprintf("  Remaining missing values: %d\n", sum(is.na(baked_train))))
```

```
Recipe successfully baked on training fold:
    Predictors processed: 4000 (Zero-variance filtered: 0)
    Post-downsample class balance: 36 Control, 30 Tumor
    Remaining missing values: 0
```

Listing 23.2 balances classes using `step_downsample` with an under-sampling ratio of 1.2. Downsampling discards 24 control samples, reducing the training set from 90 to 66 patients. In small cohorts, discarding samples reduces statistical power. Synthetic over-sampling via `themis::step_smote` generates artificial minority cases by interpolating between k-nearest neighbors; SMOTE must execute strictly inside the cross-validation training fold to prevent leakage (Deadly Sin 3). A third strategy is cost-sensitive loss weighting, which retains all training samples without synthesizing artificial points. In XGBoost, setting `scale_pos_weight = 2.0` doubles the loss penalty on misclassified tumor cases. In `ranger`, passing `case_weights` prioritizes minority observations during split evaluation without altering sample geometry. When scaling to untargeted metabolomics ($p \geq 10^4$), unsupervised filtering (such as `step_nzv` or variance thresholding) should precede model fitting in the recipe to reduce feature dimension without label leakage.

R Lab Listing 3: Multi-Model Specification and 5-Fold Cross-Validation Tuning [R 4.6 / Bioconductor]

```
# Model specifications
spec_enet <- logistic_reg(penalty = tune(), mixture = 0.5) |>
  set_engine("glmnet") |>
  set_mode("classification")

spec_rf <- rand_forest(trees = 250, min_n = tune()) |>
  set_engine("ranger", importance = "permutation", seed = 42) |>
  set_mode("classification")

spec_xgb <- boost_tree(trees = 100, tree_depth = 3, learn_rate = tune()) |>
  set_engine("xgboost", nthread = 1) |>
  set_mode("classification")

# Workflow containers
```

```
wf_enet <- workflow() |> add_recipe(rec) |> add_model(spec_enet)
wf_rf   <- workflow() |> add_recipe(rec) |> add_model(spec_rf)
wf_xgb  <- workflow() |> add_recipe(rec) |> add_model(spec_xgb)

# 5-fold stratified cross-validation on training partition
set.seed(42)
cv_folds <- vfold_cv(train_data, v = 5, strata = Status)

res_enet <- tune_grid(wf_enet, resamples = cv_folds,
                      grid = tibble(penalty = 10^seq(-3, -0.2, length.out = 6)),
                      metrics = metric_set(roc_auc, accuracy, brier_class))

res_rf <- tune_grid(wf_rf, resamples = cv_folds,
                    grid = tibble(min_n = c(3, 7, 12)),
                    metrics = metric_set(roc_auc, accuracy, brier_class))

res_xgb <- tune_grid(wf_xgb, resamples = cv_folds,
                     grid = tibble(learn_rate = c(0.02, 0.08, 0.15)),
                     metrics = metric_set(roc_auc, accuracy, brier_class))

cat("Hyperparameter grid tuning complete across all 3 model architectures.\n")
```

```
Hyperparameter grid tuning complete across all 3 model architectures.
```

R Lab Listing 4: Model Benchmark on Independent Holdout Test Set [R 4.6 / Bioconductor]

```
# Select optimal hyperparameter combinations based on CV ROC-AUC
best_enet <- select_best(res_enet, metric = "roc_auc")
best_rf   <- select_best(res_rf,   metric = "roc_auc")
best_xgb  <- select_best(res_xgb,  metric = "roc_auc")

# Fit on training partition and evaluate on untouched holdout test partition
fit_enet <- last_fit(finalize_workflow(wf_enet, best_enet), split_obj)
fit_rf   <- last_fit(finalize_workflow(wf_rf,   best_rf),   split_obj)
fit_xgb  <- last_fit(finalize_workflow(wf_xgb,  best_xgb),  split_obj)

m_enet <- collect_metrics(fit_enet)
m_rf   <- collect_metrics(fit_rf)
m_xgb  <- collect_metrics(fit_xgb)

cat("Model Benchmark on Independent Holdout Test Set (N = 30):\n")
cat(sprintf("  Elastic Net:   ROC-AUC = %.3f | Accuracy = %.3f (Optimal penalty = %.4f)\n",
            m_enet$.estimate[m_enet$.metric == "roc_auc"],
            m_enet$.estimate[m_enet$.metric == "accuracy"],
            best_enet$penalty))
cat(sprintf("  Random Forest: ROC-AUC = %.3f | Accuracy = %.3f (Optimal min_n = %d)\n",
            m_rf$.estimate[m_rf$.metric == "roc_auc"],
            m_rf$.estimate[m_rf$.metric == "accuracy"],
            best_rf$min_n))
cat(sprintf("  XGBoost:       ROC-AUC = %.3f | Accuracy = %.3f (Optimal learn_rate = %.3f)\n",
            m_xgb$.estimate[m_xgb$.metric == "roc_auc"],
            m_xgb$.estimate[m_xgb$.metric == "accuracy"],
            best_xgb$learn_rate))
```

```
Model Benchmark on Independent Holdout Test Set (N = 30):
    Elastic Net: ROC-AUC = 0.985 | Accuracy = 0.867 (Optimal penalty = 0.0010)
    Random Forest: ROC-AUC = 0.810 | Accuracy = 0.667 (Optimal min_n = 7)
    XGBoost: ROC-AUC = 0.885 | Accuracy = 0.800 (Optimal learn_rate = 0.150)
```

R Lab Listing 5: Leakage-Safe Probability Calibration Audit and Expected Calibration Error [R 4.6 / Bioconductor]

```r
ctrl_oof <- control_resamples(save_pred = TRUE)
wf_enet_final <- finalize_workflow(wf_enet, best_enet)
wf_rf_final   <- finalize_workflow(wf_rf,   best_rf)

# Fit calibration maps strictly on training out-of-fold (OOF) predictions
set.seed(42)
oof_enet <- fit_resamples(wf_enet_final, resamples = cv_folds,
                          control = ctrl_oof, metrics = metric_set(roc_auc, brier_class))
set.seed(42)
oof_rf   <- fit_resamples(wf_rf_final,   resamples = cv_folds,
                          control = ctrl_oof, metrics = metric_set(roc_auc, brier_class))

oof_pred_enet <- collect_predictions(oof_enet)
oof_pred_rf   <- collect_predictions(oof_rf)

cal_map_enet <- cal_estimate_logistic(oof_pred_enet, truth = Status)
cal_map_rf   <- cal_estimate_logistic(oof_pred_rf,   truth = Status)

# Apply frozen calibration maps to untouched holdout test partition
pred_enet_test <- collect_predictions(fit_enet)
pred_rf_test   <- collect_predictions(fit_rf)

pred_enet_cal <- cal_apply(pred_enet_test, cal_map_enet)
pred_rf_cal   <- cal_apply(pred_rf_test,   cal_map_rf)

brier_raw_enet <- brier_class(pred_enet_test, truth = Status, .pred_Control)$.estimate
brier_cal_enet <- brier_class(pred_enet_cal,  truth = Status, .pred_Control)$.estimate
brier_raw_rf   <- brier_class(pred_rf_test,   truth = Status, .pred_Control)$.estimate
brier_cal_rf   <- brier_class(pred_rf_cal,    truth = Status, .pred_Control)$.estimate

compute_ece <- function(pred_df, n_bins = 5) {
  breaks <- seq(0, 1, length.out = n_bins + 1)
  df <- pred_df |>
    mutate(bin = cut(.pred_Control, breaks = breaks, include.lowest = TRUE),
           y_num = as.numeric(Status == "Control")) |>
    group_by(bin) |>
    summarise(n = n(), conf = mean(.pred_Control), acc = mean(y_num), .groups = "drop")
  sum((df$n / nrow(pred_df)) * abs(df$acc - df$conf))
}

cat("Probability Calibration Audit (Brier Score and ECE, leakage-safe):\n")
cat(sprintf("  Elastic Net:   Raw BS = %.4f (ECE = %.3f) -> Calibrated BS = %.4f (ECE = %.3f)\n",
            brier_raw_enet, compute_ece(pred_enet_test), brier_cal_enet, compute_ece(pred_enet_cal)))
cat(sprintf("  Random Forest: Raw BS = %.4f (ECE = %.3f) -> Calibrated BS = %.4f (ECE = %.3f)\n",
            brier_raw_rf, compute_ece(pred_rf_test), brier_cal_rf, compute_ece(pred_rf_cal)))

# Holdout reliability audit for Elastic Net across probability bins
bin_breaks <- seq(0, 1, length.out = 6)
cal_audit_enet <- pred_enet_cal |>
```

```
  mutate(bin = cut(.pred_Control, breaks = bin_breaks, include.lowest = TRUE)) |>
  group_by(bin) |>
  summarise(mean_pred = round(mean(.pred_Control), 3),
            mean_obs  = round(mean(Status == "Control"), 3),
            n = n(), .groups = "drop")
print(cal_audit_enet)
```

```
Probability Calibration Audit (Brier Score and ECE, leakage-safe):
    Elastic Net: Raw BS = 0.0799 (ECE = 0.136) -> Calibrated BS = 0.1266 (ECE = 0.113)
    Random Forest: Raw BS = 0.1995 (ECE = 0.111) -> Calibrated BS = 0.1816 (ECE = 0.168)
  bin mean_pred mean_obs n
1 [0,0.2] 0.192 0.000 2
2 (0.2,0.4] 0.288 0.000 2
3 (0.4,0.6] 0.498 0.000 1
4 (0.6,0.8] 0.674 0.000 2
5 (0.8,1] 0.895 0.870 23
```

Platt scaling reduced the Expected Calibration Error for Elastic Net (ECE 0.136 $\rightarrow$ 0.113). For Random Forest, however, logistic calibration degraded ECE from 0.111 to 0.168. Tree ensemble vote fractions are bounded and central; fitting an unregularized logistic sigmoid can over-correct on small out-of-fold samples. Consequently, uncalibrated raw probabilities are retained for Random Forest in subsequent decision curve evaluations.

R Lab Listing 6: Illustrative Decision Curve Analysis and Multi-Method Feature Importance [R 4.6 / Bioconductor]

```
# Pedagogical Decision Curve Analysis on simulated holdout
dca_df <- data.frame(
  Status    = as.numeric(pred_enet_test$Status == "Tumor"),
  pred_enet = pred_enet_cal$.pred_Tumor,
  pred_rf   = pred_rf_test$.pred_Tumor
)

dca_obj <- dcurves::dca(
  Status ~ pred_enet + pred_rf,
  data = dca_df,
  thresholds = c(0.15, 0.25, 0.35, 0.50)
)
nb_table <- as.data.frame(dca_obj$dca)

cat("Illustrative Simulation DCA (Net Benefit across Decision Thresholds pt):\n")
for (thresh in c(0.15, 0.25, 0.35, 0.50)) {
  sub_nb <- subset(nb_table, abs(threshold - thresh) < 0.01)
  nb_e <- sub_nb$net_benefit[sub_nb$variable == "pred_enet"]
  nb_r <- sub_nb$net_benefit[sub_nb$variable == "pred_rf"]
  nb_all <- sub_nb$net_benefit[sub_nb$variable == "all"]
  cat(sprintf("  pt = %.2f: NetBenefit(ENet) = %.3f | NetBenefit(RF) = %.3f | Treat-All = %.3f\n",
              thresh, nb_e, nb_r, nb_all))
}

# 1. Random Forest permutation importance
rf_parsnip <- fit_rf$.workflow[[1]] |> extract_fit_parsnip()
rf_imp <- sort(ranger::importance(rf_parsnip$fit), decreasing = TRUE)
```

```
# 2. Elastic Net coefficients
enet_parsnip <- fit_enet$.workflow[[1]] |> extract_fit_parsnip()
enet_coefs <- as.matrix(coef(enet_parsnip$fit, s = best_enet$penalty))
non_zero_genes <- rownames(enet_coefs)[enet_coefs[, 1] != 0 & rownames(enet_coefs) != "(Intercept)"]

# 3. XGBoost exact TreeSHAP attributions on holdout test partition
xgb_fit_raw <- extract_fit_engine(fit_xgb$.workflow[[1]])
baked_test  <- bake(prep_rec, new_data = test_data)
test_mat    <- as.matrix(baked_test |> select(starts_with("PROT_")))
shap_contrib <- predict(xgb_fit_raw, newdata = test_mat, predcontrib = TRUE)
mean_abs_shap <- colMeans(abs(shap_contrib[, colnames(test_mat)]))
top_shap <- sort(mean_abs_shap, decreasing = TRUE)

cat("\nBiomarker Signature Recovery Audit (Multi-Method Comparison):\n")
cat(sprintf("  Elastic Net Total Features Retained: %d (Module 1 Overlap: %d/8)\n",
            length(non_zero_genes), sum(paste0("PROT_", sprintf("%04d", 1:8)) %in% non_zero_genes)))
cat(sprintf("  Random Forest Top 5 Features (Permutation Imp): %s\n", paste(names(rf_imp)[1:5], collapse = ",
    ")))
cat(sprintf("  XGBoost Top 5 Features (Exact TreeSHAP):        %s\n", paste(names(top_shap)[1:5], collapse = ",
    ")))
```

```
Illustrative Simulation DCA (Net Benefit across Decision Thresholds pt):
    pt = 0.15: NetBenefit(ENet) = 0.267 | NetBenefit(RF) = 0.216 | Treat-All = 0.216
    pt = 0.25: NetBenefit(ENet) = 0.233 | NetBenefit(RF) = 0.111 | Treat-All = 0.111
    pt = 0.35: NetBenefit(ENet) = 0.200 | NetBenefit(RF) = 0.049 | Treat-All = -0.026
    pt = 0.50: NetBenefit(ENet) = 0.167 | NetBenefit(RF) = -0.000 | Treat-All = -0.333
Biomarker Signature Recovery Audit (Multi-Method Comparison):
    Elastic Net Total Features Retained: 126 (Module 1 Overlap: 8/8)
    Random Forest Top 5 Features (Permutation Imp): PROT_0106, PROT_0206, PROT_0215, PROT_0093, PROT_0070
    XGBoost Top 5 Features (Exact TreeSHAP): PROT_0215, PROT_0093, PROT_1613, PROT_0114, PROT_0176
```

23.4.1 Batch-Aware Resampling and Sampling Uncertainty Audit

When biological classes correlate with acquisition batches (Cramér's $V = 0.637$), standard cross-validation permits tree ensembles to memorize batch-shifted background features, yielding over-optimistic cross-validation performance. Listing 23.7 evaluates performance using Leave-One-Batch-Out Cross-Validation (LOBOCV), forcing each evaluation fold to comprise an unseen analytical batch.

R Lab Listing 7: Batch-Aware Resampling via Leave-One-Batch-Out Cross-Validation [R 4.6 / Bioconductor]

```
# Tabulate batch-class confounding association
tab_bc <- table(train_data$Batch, train_data$Status)
cramer_v <- sqrt(chisq.test(tab_bc)$statistic / (sum(tab_bc) * (min(nrow(tab_bc), ncol(tab_bc)) - 1)))
cat(sprintf("Batch-class confounding association (Cramer's V): %.3f\n", as.numeric(cramer_v)))

# Construct Leave-One-Batch-Out folds
lobo_folds <- group_vfold_cv(train_data, group = "Batch", v = NULL)

rec_lobo <- recipe(Status ~ ., data = train_data) |>
  update_role(Batch, new_role = "batch_id") |>
  update_role(Age, new_role = "imputation donor") |>
  step_impute_median(all_numeric_predictors()) |>
  step_zv(all_numeric_predictors()) |>
```

```
  step_normalize(all_numeric_predictors())

wf_enet_lobo <- workflow() |> add_recipe(rec_lobo) |> add_model(spec_enet)
wf_rf_lobo   <- workflow() |> add_recipe(rec_lobo) |> add_model(spec_rf)
wf_xgb_lobo  <- workflow() |> add_recipe(rec_lobo) |> add_model(spec_xgb)

set.seed(42)
res_enet_lobo <- fit_resamples(finalize_workflow(wf_enet_lobo, best_enet),
                               resamples = lobo_folds, control = control_resamples(save_pred = TRUE),
                               metrics = metric_set(roc_auc, accuracy, brier_class))
set.seed(42)
res_rf_lobo   <- fit_resamples(finalize_workflow(wf_rf_lobo, best_rf),
                               resamples = lobo_folds, control = control_resamples(save_pred = TRUE),
                               metrics = metric_set(roc_auc, accuracy, brier_class))
set.seed(42)
res_xgb_lobo  <- fit_resamples(finalize_workflow(wf_xgb_lobo, best_xgb),
                               resamples = lobo_folds, control = control_resamples(save_pred = TRUE),
                               metrics = metric_set(roc_auc, accuracy, brier_class))

cat("Resampling Architecture Comparison (Batch Confounder Impact):\n")
cat(sprintf("  Elastic Net:   Stratified CV AUC = %.3f  ->  LOBOCV AUC = %.3f (Drop = %+.3f)\n",
            collect_metrics(res_enet) |> filter(.metric == "roc_auc") |> pull(mean),
            collect_metrics(res_enet_lobo) |> filter(.metric == "roc_auc") |> pull(mean),
            (collect_metrics(res_enet_lobo) |> filter(.metric == "roc_auc") |> pull(mean)) -
            (collect_metrics(res_enet) |> filter(.metric == "roc_auc") |> pull(mean))))
cat(sprintf("  Random Forest: Stratified CV AUC = %.3f  ->  LOBOCV AUC = %.3f (Drop = %+.3f)\n",
            collect_metrics(res_rf) |> filter(.metric == "roc_auc") |> pull(mean),
            collect_metrics(res_rf_lobo) |> filter(.metric == "roc_auc") |> pull(mean),
            (collect_metrics(res_rf_lobo) |> filter(.metric == "roc_auc") |> pull(mean)) -
            (collect_metrics(res_rf) |> filter(.metric == "roc_auc") |> pull(mean))))
cat(sprintf("  XGBoost:       Stratified CV AUC = %.3f  ->  LOBOCV AUC = %.3f (Drop = %+.3f)\n",
            collect_metrics(res_xgb) |> filter(.metric == "roc_auc") |> pull(mean),
            collect_metrics(res_xgb_lobo) |> filter(.metric == "roc_auc") |> pull(mean),
            (collect_metrics(res_xgb_lobo) |> filter(.metric == "roc_auc") |> pull(mean)) -
            (collect_metrics(res_xgb) |> filter(.metric == "roc_auc") |> pull(mean))))
```

```
Batch-class confounding association (Cramér's V): 0.637
Resampling Architecture Comparison (Batch Confounder Impact):
    Elastic Net: Stratified CV AUC = 0.828 -> LOBOCV AUC = 0.785 (Drop = -0.043)
    Random Forest: Stratified CV AUC = 0.825 -> LOBOCV AUC = 0.584 (Drop = -0.241)
    XGBoost: Stratified CV AUC = 0.731 -> LOBOCV AUC = 0.517 (Drop = -0.214)
```

On small holdout cohorts ($N = 30$), marginal performance metrics carry wide estimation uncertainty. Listing 23.8 computes non-parametric bootstrap percentile confidence intervals ($B = 2{,}000$) alongside paired bootstrap difference tests ($\Delta_{\text{AUC}} = \text{AUC}_{\text{model}_1} - \text{AUC}_{\text{model}_2}$) evaluated across the same 30 holdout patients. Stating that architectures are equivalent based on overlapping marginal confidence intervals is an inferential fallacy [21]; paired resampling evaluates whether difference distributions exclude zero. Under paired bootstrap testing, Elastic Net outperformed Random Forest (mean $\Delta_{\text{AUC}} = +0.175$, 95% CI $[+0.030, +0.381]$, paired $p = 0.014$) and XGBoost (mean $\Delta_{\text{AUC}} = +0.099$, 95% CI $[+0.004, +0.240]$, paired $p = 0.024$). However, the wide marginal intervals (e.g., Random Forest AUC $[0.608, 0.963]$) emphasize that holdout sizes of $N = 30$ carry substantial sampling variability, precluding definitive architectural ranking without external validation cohorts [11].

R Lab Listing 8: Paired Bootstrap Resampling and Estimation Uncertainty Audit [R 4.6 / Bioconductor]

```
test_preds <- list(
  ENet = collect_predictions(fit_enet) |> select(.row, Status, .pred_class, .pred_Control),
  RF   = collect_predictions(fit_rf)   |> select(.row, Status, .pred_class, .pred_Control),
  XGB  = collect_predictions(fit_xgb)  |> select(.row, Status, .pred_class, .pred_Control)
)

set.seed(42)
B <- 2000
N_test <- nrow(test_preds$ENet)

boot_paired <- map_dfr(1:B, function(i) {
  idx <- sample.int(N_test, size = N_test, replace = TRUE)
  df_e <- test_preds$ENet[idx, ]; df_r <- test_preds$RF[idx, ]; df_x <- test_preds$XGB[idx, ]
  if (length(unique(df_e$Status)) < 2) return(NULL)

  auc_e <- roc_auc(df_e, truth = Status, .pred_Control)$.estimate
  auc_r <- roc_auc(df_r, truth = Status, .pred_Control)$.estimate
  auc_x <- roc_auc(df_x, truth = Status, .pred_Control)$.estimate

  tibble(auc_e = auc_e, auc_r = auc_r, auc_x = auc_x,
         delta_er = auc_e - auc_r, delta_ex = auc_e - auc_x,
         acc_e = accuracy(df_e, truth = Status, estimate = .pred_class)$.estimate,
         acc_r = accuracy(df_r, truth = Status, estimate = .pred_class)$.estimate,
         acc_x = accuracy(df_x, truth = Status, estimate = .pred_class)$.estimate,
         br_e  = brier_class(df_e, truth = Status, .pred_Control)$.estimate,
         br_r  = brier_class(df_r, truth = Status, .pred_Control)$.estimate,
         br_x  = brier_class(df_x, truth = Status, .pred_Control)$.estimate)
})

cat("Marginal 95% Confidence Intervals (N = 30, B = 2000):\n")
cat(sprintf("  ENet: AUC = %.3f [%.3f, %.3f] | Acc = %.3f [%.3f, %.3f] | Brier = %.3f [%.3f, %.3f]\n",
            m_enet$.estimate[m_enet$.metric == "roc_auc"], quantile(boot_paired$auc_e, 0.025),
     quantile(boot_paired$auc_e, 0.975),
            m_enet$.estimate[m_enet$.metric == "accuracy"], quantile(boot_paired$acc_e, 0.025),
     quantile(boot_paired$acc_e, 0.975),
            brier_class(test_preds$ENet, truth = Status, .pred_Control)$.estimate, quantile(boot_paired$br_e,
     0.025), quantile(boot_paired$br_e, 0.975)))
cat(sprintf("  RF:   AUC = %.3f [%.3f, %.3f] | Acc = %.3f [%.3f, %.3f] | Brier = %.3f [%.3f, %.3f]\n",
            m_rf$.estimate[m_rf$.metric == "roc_auc"], quantile(boot_paired$auc_r, 0.025),
     quantile(boot_paired$auc_r, 0.975),
            m_rf$.estimate[m_rf$.metric == "accuracy"], quantile(boot_paired$acc_r, 0.025),
     quantile(boot_paired$acc_r, 0.975),
```

```
            brier_class(test_preds$RF, truth = Status, .pred_Control)$.estimate, quantile(boot_paired$br_r,
    0.025), quantile(boot_paired$br_r, 0.975)))
cat(sprintf("  XGB:  AUC = %.3f [%.3f, %.3f] | Acc = %.3f [%.3f, %.3f] | Brier = %.3f [%.3f, %.3f]\n",
            m_xgb$.estimate[m_xgb$.metric == "roc_auc"], quantile(boot_paired$auc_x, 0.025),
    quantile(boot_paired$auc_x, 0.975),
            m_xgb$.estimate[m_xgb$.metric == "accuracy"], quantile(boot_paired$acc_x, 0.025),
    quantile(boot_paired$acc_x, 0.975),
            brier_class(test_preds$XGB, truth = Status, .pred_Control)$.estimate, quantile(boot_paired$br_x,
    0.025), quantile(boot_paired$br_x, 0.975)))

cat("\nPaired Bootstrap Difference Tests (Delta AUC):\n")
cat(sprintf("  ENet vs RF:  Mean Delta = %+.3f, 95%% CI [%+.3f, %+.3f], p = %.3f\n",
            mean(boot_paired$delta_er), quantile(boot_paired$delta_er, 0.025), quantile(boot_paired$delta_er,
    0.975),
            2 * min(mean(boot_paired$delta_er > 0), mean(boot_paired$delta_er < 0))))
cat(sprintf("  ENet vs XGB: Mean Delta = %+.3f, 95%% CI [%+.3f, %+.3f], p = %.3f\n",
            mean(boot_paired$delta_ex), quantile(boot_paired$delta_ex, 0.025), quantile(boot_paired$delta_ex,
    0.975),
            2 * min(mean(boot_paired$delta_ex > 0), mean(boot_paired$delta_ex < 0))))
```

```
Marginal 95% Confidence Intervals (N = 30, B = 2000):
    ENet: AUC = 0.985 [0.937, 1.000] | Acc = 0.867 [0.733, 0.967] | Brier = 0.080 [0.031, 0.140]
    RF: AUC = 0.810 [0.608, 0.963] | Acc = 0.667 [0.500, 0.833] | Brier = 0.200 [0.169, 0.234]
    XGB: AUC = 0.885 [0.740, 0.981] | Acc = 0.800 [0.667, 0.933] | Brier = 0.127 [0.062, 0.201]
Paired Bootstrap Difference Tests (Delta AUC):
    ENet vs RF: Mean Delta = +0.175, 95% CI [+0.030, +0.381], p = 0.014
    ENet vs XGB: Mean Delta = +0.099, 95% CI [+0.004, +0.240], p = 0.024
```

Computational Environment and Reproducibility Parameters

The code listings in this laboratory were executed and validated under R 4.6.1 (2026-06-24 ucrt). Key package dependencies: `tidymodels` 1.2.0, `glmnet` 4.1-8, `ranger` 0.16.0, `xgboost` 1.7.8.1, `themis` 1.0.2, `probably` 1.0.3, `dcurves` 0.4.0, and `vcd` 1.4-14. Random seeds are fixed via `set.seed(2026)` (cohort simulation) and `set.seed(42)` (resampling splits) to ensure numerical reproduction across platforms. Complete reproducible scripts are available in `code/chapter23_revised_labs.R`.

23.5 The Five Deadly Sins of Omics Machine Learning

Methodological Pitfall & Deadly Sin

Deadly Sin 1: Pre-Filtering and Batch Correction Data Leakage.
Mechanism: Screening features via univariate tests or fitting batch normalization models (such as ComBat or RUV) across all N patients before train/test splitting.
Consequence: Validation set labels and variance parameters leak into the training pipeline, yielding 99% cross-validation accuracy that fails on independent instrument runs [1, 7].
Remedy: Execute all feature filtering, scaling, imputation, and batch parameter fitting strictly inside the cross-validation training fold (`tidymodels::recipe`), and validate using Leave-One-Batch-Out Cross-Validation (LOBOCV) [17].

Methodological Pitfall & Deadly Sin

Deadly Sin 2: Using Default LASSO ($\alpha = 1$) on Strongly Collinear Omics Data.
Mechanism: Applying standard LASSO to co-eluting peptides or tightly co-regulated metabolites.
Consequence: LASSO arbitrarily picks one feature and zeroes out all biological partners, leading to erratic signatures that vary between cross-validation folds [28].
Remedy: Deploy the Elastic Net with $\alpha \in [0.2, 0.8]$ to enforce the grouping property (Eq. 23.9).

Methodological Pitfall & Deadly Sin

Deadly Sin 3: Applying SMOTE to the Entire Dataset Before Resampling.
Mechanism: Generating synthetic minority samples using SMOTE on the complete dataset before cross-validation.
Consequence: Synthetic test points are derived directly from training neighbors, causing over-fitting.
Remedy: Apply SMOTE strictly to the training fold within the resampling recipe (`themis::step_smote`) [8], or use cost-sensitive loss weights (`scale_pos_weight`) to avoid discarding data or synthesizing artificial points.

Methodological Pitfall & Deadly Sin

Deadly Sin 4: Reporting Unnested Cross-Validation Performance.
Mechanism: Tuning hyperparameters on a cross-validation grid and reporting the best-tuned performance as the model's true accuracy.
Consequence: Hyperparameter tuning on validation folds introduces optimistic selection bias.
Remedy: Deploy nested cross-validation (5×5) or an independent untouched holdout test partition [16].

Methodological Pitfall & Deadly Sin

Deadly Sin 5: Trusting Uncalibrated Probabilities and Neglecting Clinical Net Benefit.
Mechanism: Interpreting raw Random Forest vote fractions or SVM margins as absolute clinical risk probabilities without auditing calibration.
Consequence: Probabilities cluster away from 0 and 1, distorting clinical decision thresholds and risking harmful patient interventions.
Remedy: Audit Brier scores, plot reliability diagrams, apply Platt scaling, and evaluate Decision Curve Analysis [6, 26].

23.6 Tutor's Self-Assessment Diagnostic Checklist

To confirm the methodological validity of your machine learning workflow prior to reporting clinical biomarker signatures, execute the diagnostic audit summarized in Table 23.2.

Table 23.2: Tutor's Self-Assessment Diagnostic Checklist for Omics Machine Learning

Step	Diagnostic Metric	Acceptable Threshold	Corrective Protocol
1	Leakage Audit	Zero preprocessing or batch fitting prior to split.	Encapsulate all steps inside `recipe` [17].
2	Grouping Bound Check	Collinear biomarker blocks retained collectively.	Shift from LASSO to Elastic Net ($\alpha = 0.5$) [28].
3	Resampling Architecture	Nested CV, holdout set ($N \geq 20$), or LOBOCV.	Eliminate tuning and batch bias; report outer test AUC [16].
4	Probability Calibration	Expected Calibration Error ECE ≤ 0.15; Brier drop.	Apply Platt scaling or isotonic calibration [20].
5	Clinical Net Benefit	DCA demonstrates positive net benefit over Treat-All.	Benchmark against treat-all / treat-none [26].
6	Feature Permutation Check	True modules rank above empirical null distribution.	Compute permutation importance or TreeSHAP [19].

23.7 Chapter Exercises

Exercise 23.1 (Elastic Net Grouping Bound Calculation):
Two co-eluting biomarker peptides have a sample correlation of $\rho = 0.98$. An Elastic Net model is fitted with penalty parameter $\lambda = 0.50$ and mixture $\alpha = 0.40$. The root-mean-squared error of the fit is RMSE $= 0.80$.

1. Compute the theoretical upper bound on the coefficient difference $|\hat{\beta}_1 - \hat{\beta}_2|$ using Eq. (23.9).
2. If the correlation drops to $\rho = 0.50$, what is the new upper bound? Explain the biological implication of this difference.

Exercise 23.2 (Probability Calibration and Brier Score Decomposition):
Consider $n = 5$ patients with predicted disease probabilities $\hat{\mathbf{p}} = (0.10, 0.30, 0.60, 0.85, 0.90)^T$ and true clinical outcomes $\mathbf{y} = (0, 0, 1, 1, 1)^T$.

1. Compute the empirical Brier score BS $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{p}_i)^2$.
2. Compare this to a naive uninformative classifier that predicts $\hat{p} = 0.50$ for all patients.

Exercise 23.3 (Clinical Decision Curve Analysis Net Benefit Derivation):
A clinical validation cohort has disease prevalence $P(Y = 1) = 0.30$ ($N = 100$ patients: 30 cases, 70 non-cases). A proteomic signature evaluated at decision threshold $p_t = 0.25$ identifies 24 True Positives, 12 False Positives, 58 True Negatives, and 6 False Negatives.

1. Compute the clinical Net Benefit of the proteomic signature at $p_t = 0.25$.
2. Compute the Net Benefit of the default clinical strategy "Treat All Patients" at $p_t = 0.25$.
3. Compare the model to "Treat All" and "Treat None", and state the clinical utility.

23.8 Worked-Out Numerical Solutions

Solution to Exercise 23.1: Elastic Net Grouping Bound

1. Upper Bound for $\rho = 0.98$:
From Eq. (23.9):

$$|\hat{\beta}_1 - \hat{\beta}_2| \leq \frac{\sqrt{2(1-\rho)}}{\lambda(1-\alpha)} \cdot \text{RMSE}$$

Given $\rho = 0.98$, $\lambda = 0.50$, $\alpha = 0.40$, RMSE $= 0.80$:

$$\sqrt{2(1-0.98)} = \sqrt{2(0.02)} = \sqrt{0.04} = 0.20$$

$$\lambda(1-\alpha) = 0.50(1-0.40) = 0.50(0.60) = 0.30$$

$$|\hat{\beta}_1 - \hat{\beta}_2| \leq \frac{0.20}{0.30} \times 0.80 = \frac{2}{3} \times 0.80 \approx 0.533$$

2. Upper Bound for $\rho = 0.50$:

$$\sqrt{2(1-0.50)} = \sqrt{2(0.50)} = \sqrt{1.0} = 1.00$$

$$|\hat{\beta}_1 - \hat{\beta}_2| \leq \frac{1.00}{0.30} \times 0.80 = 3.333 \times 0.80 \approx 2.667$$

Biological Interpretation: For correlated biomarkers ($\rho = 0.98$), the Elastic Net guarantees that coefficients cannot differ by more than 0.533. When correlation drops to $\rho = 0.50$, the bound relaxes to 2.667 (5× wider), allowing the model to decouple features with independent biological trajectories [28].

Solution to Exercise 23.2: Brier Score Computation

1. Brier Score of Classifier:
Individual squared errors:

$$(y_1 - \hat{p}_1)^2 = (0 - 0.10)^2 = 0.0100$$
$$(y_2 - \hat{p}_2)^2 = (0 - 0.30)^2 = 0.0900$$
$$(y_3 - \hat{p}_3)^2 = (1 - 0.60)^2 = 0.1600$$
$$(y_4 - \hat{p}_4)^2 = (1 - 0.85)^2 = 0.0225$$
$$(y_5 - \hat{p}_5)^2 = (1 - 0.90)^2 = 0.0100$$

Sum of squared errors:

$$\sum_{i=1}^{5} (y_i - \hat{p}_i)^2 = 0.0100 + 0.0900 + 0.1600 + 0.0225 + 0.0100 = 0.2925$$

$$\text{BS} = \frac{0.2925}{5} = 0.0585$$

2. Comparison to Naive Benchmark:
For $\hat{p} \equiv 0.50$:

$$(0-0.5)^2 \times 2 + (1-0.5)^2 \times 3 = 0.25 \times 5 = 1.25 \implies \text{BS}_{\text{naive}} = \frac{1.25}{5} = 0.2500$$

The trained classifier achieves a Brier score of 0.0585, a 76.6% reduction in prediction error compared to the uninformative baseline (0.2500), demonstrating probability calibration [6].

Solution to Exercise 23.3: Decision Curve Analysis Net Benefit

1. Net Benefit of Proteomic Model at $p_t = 0.25$:
From Vickers and Elkin [26]:

$$\text{Net Benefit} = \frac{\text{TP}}{N} - \frac{\text{FP}}{N}\left(\frac{p_t}{1-p_t}\right)$$

Given $N = 100$, TP $= 24$, FP $= 12$, $p_t = 0.25$:

$$\frac{p_t}{1-p_t} = \frac{0.25}{1-0.25} = \frac{0.25}{0.75} = \frac{1}{3}$$

$$\text{Net Benefit} = \frac{24}{100} - \frac{12}{100}\left(\frac{1}{3}\right) = 0.240 - 0.040 = 0.200$$

2. Net Benefit of Default "Treat All":
If all 100 patients receive treatment, TP $= 30$ and FP $= 70$:

$$\text{Net Benefit}_{\text{All}} = \frac{30}{100} - \frac{70}{100}\left(\frac{1}{3}\right) = 0.300 - 0.2333 = 0.0667$$

3. Clinical Utility Comparison:
For "Treat None", TP $= 0$ and FP $= 0$, giving $\text{Net Benefit}_{\text{None}} = 0.000$. The model's net benefit of 0.200 exceeds both Treat-All (0.067) and Treat-None (0.000). The net benefit gain over Treat-All is $0.200 - 0.067 = 0.133$, which corresponds to detecting 13.3 additional true cancer cases per 100 patients without increasing false-positive interventions.

Bibliography

[1] Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10), 6562--6566. doi:10.1073/pnas.102102699.

[2] Barker, M., & Rayens, W. (2003). Partial least squares for discrimination. *Journal of Chemometrics*, 17(3), 166--173. doi:10.1002/cem.785.

[3] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1), 289--300. doi:10.1111/j.2517-6161.1995.tb02031.x.

[4] Boland, M. R. (2023). *Health Data Analytics with R: A Hands-On Guide*. Springer Nature, Cham, Switzerland. doi:10.1007/978-3-031-18099-6.

[5] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5--32. doi:10.1023/A:1010933404324.

[6] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1--3. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[7] Broadhurst, D. I., & Kell, D. B. (2006). Statistical strategies for avoiding false discoveries in metabolomics and related systems biology. *Metabolomics*, 2(4), 171--196. doi:10.1007/s11306-006-0037-z.

[8] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321--357. doi:10.1613/jair.953.

[9] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785--794). doi:10.1145/2939672.2939785.

[10] Cover, T. M. (1965). Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition. *IEEE Transactions on Electronic Computers*, EC-14(3), 326--334. doi:10.1109/PGEC.1965.264137.

[11] DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3), 837--845. doi:10.2307/2531595.

[12] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189--1232. doi:10.1214/aos/1013203451.

[13] Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1--22. doi:10.18637/jss.v033.i01.

[14] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer, New York, NY.

[15] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55--67. doi:10.1080/00401706.1970.10488634.

[16] Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer, New York, NY. doi:10.1007/978-1-4614-6849-3.

[17] Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly Media, Sebastopol, CA.

[18] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765--4774.

[19] Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S. I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56--67. doi:10.1038/s42256-019-0138-9.

[20] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3), 61--74.

[21] Schenker, N., & Gentleman, J. F. (2001). On judging the significance of differences by examining the overlap between confidence intervals. *The American Statistician*, 55(3), 182--186. doi:10.1198/000313001317098149.

[22] Strobl, C., Boulesteix, A. L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8, 25. doi:10.1186/1471-2105-8-25.

[23] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267--288. doi:10.1111/j.2517-6161.1996.tb02080.x.

[24] Trygg, J., & Wold, S. (2002). Orthogonal projections to latent structures (O-PLS). *Journal of Chemometrics*, 16(3), 119--128. doi:10.1002/cem.695.

[25] Vapnik, V. N. (1998). *Statistical Learning Theory*. John Wiley & Sons, New York.

[26] Vickers, A. J., & Elkin, E. B. (2006). Decision curve analysis: a novel method for evaluating prediction models. *Medical Decision Making*, 26(6), 565--574. doi:10.1177/0272989X06295361.

[27] Westerhuis, J. A., Hoefsloot, H. C., Smit, S., Vis, D. J., Smilde, A. K., van Velzen, E. J., van Duynhoven, J. P., & Praticò, G. (2008). Assessment of PLSDA cross validation. *Metabolomics*, 4(1), 81--89. doi:10.1007/s11306-007-0099-6.

[28] Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2), 301--320. doi:10.1111/j.1467-9868.2005.00503.x.